1. Question 1

Django is a high-level, open-source web framework written in Python that is used for building secure, scalable, and maintainable web applications. It was developed to simplify complex web development tasks and to encourage rapid development. Django follows best software engineering practices and enforces a clean and organized project structure.

One of the most important characteristics of Django is that it follows the MVT (Model–View–Template) architecture, which separates database logic, business logic, and presentation layer. This separation improves maintainability and scalability of applications.

Django provides many built-in features such as ORM, automatic admin interface, authentication system, session management, middleware support, URL routing system, and strong security mechanisms. It protects applications against common threats like SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

The major advantages of Django include rapid development, security by default, scalability for enterprise-level applications, reusable components, and strong community support. Large companies such as Instagram and Pinterest use Django for handling high traffic applications.

2. Question 2

Django follows the Model–View–Template (MVT) architectural pattern. This architecture divides the web application into three main components to ensure clean separation of concerns.

The Model represents the database layer. It defines the data structure using Python classes. Each model class corresponds to a database table and each attribute corresponds to a table column. Models handle data validation and database interaction using ORM.

The View contains the business logic of the application. It receives HTTP requests from the user, processes data using models, and returns an appropriate HTTP response. Views act as a bridge between models and templates.

The Template represents the presentation layer. It contains HTML code combined with Django Template Language (DTL) to display dynamic data to the user. This structured approach ensures better code organization and maintainability.

3. Question 3

Django and Flask are both Python web frameworks but they differ in design philosophy and features. Django is a full-stack framework that provides built-in features such as ORM, authentication, admin panel, and security tools. Flask is a micro-framework that provides minimal core functionality and allows developers to add extensions as needed.

Django is suitable for large-scale, database-driven applications because it provides structured project architecture. Flask offers more flexibility and is lightweight, making it ideal for small applications or APIs.

In summary, Django emphasizes convention over configuration and rapid development, while Flask focuses on simplicity and flexibility.

4. Question 4

A Django project represents the complete web application including configuration, settings, and multiple apps. A Django app is a modular component within a project designed to perform a specific function such as user management or task handling. Projects can include multiple apps to maintain modular design.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

5. Question 5

After running startproject, Django creates manage.py and a project folder containing __init__.py, settings.py, urls.py, wsgi.py, and asgi.py. manage.py is used to run commands, settings.py stores configuration, urls.py handles routing, wsgi.py and asgi.py are used for deployment.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

6. Question 6

Django is installed using pip install django. A new project is created using django-admin startproject projectname. The development server is started using python manage.py runserver. This setup initializes the project environment.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

7. Question 7

django-admin is a global utility used to create new projects. manage.py is project-specific and executes commands within the project configuration.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

8. Question 8

The development server is a lightweight built-in server used for testing and development purposes. It is not recommended for production deployment.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

9. Question 9

settings.py stores configuration like database and installed apps. urls.py maps URLs. wsgi.py handles synchronous deployment. asgi.py supports asynchronous deployment.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

10. Question 10

URL dispatcher maps incoming URL requests to corresponding view functions defined in urls.py.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

11. Question 11

Function-Based Views use simple functions. Class-Based Views use Python classes and provide reusable patterns.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

12. Question 12

path() handles simple URL patterns. re_path() uses regular expressions for complex routing.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

13. Question 13

Parameters are passed using angle brackets such as in URL patterns.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

14. Question 14

HttpRequest contains client request data. HttpResponse sends response back to the client.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

15. Question 15

DTL allows dynamic content rendering using variables, loops, and conditional statements.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

16. Question 16

Template tags control logic while filters modify variable output.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

17. Question 17

Template inheritance allows reuse of common layout using base templates and block structures.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

18. Question 18

Static files include CSS, JS, and images configured using STATIC_URL.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

19. Question 19

A Django model is a Python class representing a database table managed through ORM.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

20. Question 20

makemigrations creates migration files. migrate applies them to the database.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

21. Question 21

CharField stores short text. TextField stores long text. IntegerField stores numbers. ForeignKey creates relationships.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

22. Question 22

ORM allows database interaction using Python objects instead of SQL.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

23. Question 23

Django Forms provide automatic validation and CSRF protection unlike basic HTML forms.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

24. Question 24

CSRF token protects against malicious cross-site request forgery attacks.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

25. Question 25

GET retrieves data while POST submits data using forms.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

26. Question 26

Django authentication provides login, logout, permissions, and password hashing.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

27. Question 27

Django Admin is an automatic interface to manage database records.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

28. Question 28

Static files are developer assets. Media files are user-uploaded content.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

29. Question 29

Middleware processes request and response globally.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.

30. Question 30

Django REST Framework enables building RESTful APIs using serializers and API views.

This concept is essential in real-world Django application development and commonly asked in academic examinations.

It ensures structured, secure, and maintainable project implementation when building scalable web systems.